

First subproblem

We can simply construct all binary grids of the given dimensions, for each check whether it is valid according to the rules given in the statement, and remember the smallest number of ones we had in a grid that was valid.

There are 2^{rc} binary grids, and we can validate each one in $O(rc)$, so our time complexity is $O(rc2^{rc})$ per test case.

Second subproblem

Notice that the rules about neighbors are local — whether a cell satisfies the rules does not depend on cells that are not its direct neighbors. One consequence of this is that the validity of cells in a row only depends on the cells in the neighboring rows.

Thus, we can use dynamic programming and process the grid row by row. Our state will be which row i we placed last, and the values we placed in rows i and $i - 1$ so that row $i - 1$ was valid (it has to be, since its set of neighbors is finalized). For each such state, we will remember the smallest number of 1s we had to place to reach it.

The initial state represents the first two rows. These two rows must be such that the first row matches the pattern p and the first row is valid (since it's finalized). Note that there can be multiple initial states (a pattern can match many possible rows, and each first row can have multiple possible continuations i.e. next rows).

Also note, if there is only 1 row, we have to process this case separately (but this is easy). For the rest of this writeup, we assume that $r \geq 2$.

Then, to compute the next states, try appending every possible row $i + 1$. For each one, check whether the i -th row would satisfy the rules, and if so, we get a new state (where the last two rows are the i -th one from our state, and the $i + 1$ -st we just placed), and we update the number of 1s needed to reach it, if it is smaller than what we have found for it before.

At the end, we can go over all states having placed the r -th row, check whether the r -th row itself satisfies the rules (as we won't be placing any more rows) and satisfies pattern q , and output the smallest value across all such states.

We will place $O(r)$ rows, for each one we have $O(2^{2c})$ possible values of the previous two rows, and for each such state we try appending the next row, for which we have 2^c possibilities. Checking the validity of the rows whose neighbors we finalized takes us $O(c)$, so our total complexity is $O(rc2^{3c})$ per test case.

Third subproblem

We apply a few optimizations to the dynamic programming solution. First, the actual number of possible rows and two-row-states to consider is smaller than 2^c (resp. 2^{2c}). If there is a cell containing a 1 that is adjacent to both 0 and a 1, then the constraints are already violated, and we can discard that row / two-row-state. This cuts down the number of two-row-states significantly: for $c = 7$, there are 578 of them (compared to 2^{14} originally). We also only have to consider those rows that are part of some valid two-row-state.

Second optimization is, assume we have a fixed c and a predetermined first row. Then, even if we have multiple test cases with different r , the dynamic programming starts from the same initial set of states, so it evolves the same. The only difference is in the finish; some test cases finish earlier (smaller r), some later (bigger r), and when they finish we have to consider additional constraints (i.e. there is no row after the last row, so it must already be good + it must match the pattern q , which is different between test cases). I.e. instead of calculating $r_1 + r_2 + \dots + r_i$ rows of DP, we only need to calculate $\max(r_1, \dots, r_i) = r_{max}$.

What if we don't have a fixed first row? We can precompute the DP for all possible first rows, and then given a pattern p , just search all the candidate first rows that match that pattern. It turns out the number of possible first rows isn't large at all: just by discarding those candidates that are not part of any two-row-state, we can cut down the number to around 30 for $c = 7$ (and even less for $c < 7$).

Let's analyze the time complexity. Denote the maximum number of two-row-states as s_2 and the maximum number of candidate first rows as s_1 . Then, the number of states (required for processing *all* test cases for a single c) is $O(r_{max} \cdot s_2 \cdot s_1)$. For each state, we need to try all possible extensions, and there are $O(2^c)$ of them. For each test case, we consider all $O(s_1)$ possible starting rows; for each, we consider the $O(s_2)$ final states and check that they match the pattern q in $O(c)$ time. So the total time complexity for a single c is

$$O(r_{max} \cdot s_1 \cdot s_2 \cdot 2^c + T \cdot s_1 \cdot s_2 \cdot c).$$

This can be optimized further by noting that when answering test cases, we're no longer interested in two-row-states but only in possible last rows (which there are s_1 instead of s_2). So the time complexity can be brought down to

$$O(r_{max} \cdot s_1 \cdot s_2 \cdot 2^c + T \cdot s_1^2 \cdot c).$$

This is for processing all test cases with a given c , processing all c takes at most c -times longer (but smaller c actually take less time, so this is conservative).

Subproblem 4

For large r , we can't afford to calculate the DP row-by-row.

Observation: consider the DP after processing row k . There are only s_2 two-row-states, and the DP state in row $k + 1$ is fully determined by the DP state at row k as follows. Let's index the states $1, 2, \dots, s_2$; denote the vector of values at DP k as u and the vector of values at DP $k + 1$ as v . Then,

$$v[i] = \min_{j=1, \dots, s_2} (u[j] + c_{j,i}),$$

where $c_{j,i}$ is the cost of the row that we need to append to state j to get to state i . (Cost is just the number of ones in that row.) If it is impossible to get from state j to i , that can be represented by an infinite cost.

This is just matrix multiplication, with the matrix entries being $c_{j,i}$ and we are operating over the *tropical semiring* (where addition is $\min$ and multiplication is $+$). The DP computed in earlier subproblems was just us applying the matrix to the initial state r -times. But for large r , this is inefficient, but instead of doing

$$M(M(M(\dots M \cdot u)))$$

we can do

$$(M \cdot M \cdot M \cdot \dots \cdot M) \cdot u$$

i.e. compute the r -th power of M , which is easily done in $O(s_2^3 \cdot \log r)$ time, per test case.

Final subproblem

The above is still very expensive for large number of test cases. However, we have this hunch that perhaps to minimize the number of 1s, it's best to repeat some efficient tiling a lot of times. I.e. that there is some periodicity to the DP. This is indeed the case.

Consider the matrix M ; this matrix describes the transitions between states. Consider only those states that can be returned to (the other ones are such that we can visit them only a constant number of times, regardless of r). It turns out that among these states, there are only two strongly connected components. One of them contains the single state where both rows are 111...111. (This is because if any two 1s are ever adjacent, then all their neighbors must be 1s too. This propagates to the entire grid.) The second one contains all the remaining states (that can be returned to). This means that, there are only two "efficient tilings": one where all rows are 111...111 (we may be forced to use this tiling e.g. due to p, q), and one for all the remaining states (it isn't efficient to use an 'inefficient tiling', and all the states can reach the 'efficient tiling' since they are all connected).

This is a bit hand-wavy, so let's do it properly. Say we have two state vectors u, v that differ only by a constant (in those entries that are finite, and they have the same infinite entries). Then, applying any matrix M to both, we get u' and v' , which too will differ by the same constant, i.e. the relative differences between entries are preserved. Call these relative differences the *signature*; if, during our process of applying M , we ever get two vectors with the same signature, the rest will be periodic.

Formally, denote the state vector at row i as $f(i)$. Denote the row number where the cycle starts r_0 and the row number where we first repeat it r_1 . Let $\Delta r = r_1 - r_0$. Then, for all $i \geq r_0$, it holds that

$$f(i + \Delta r) - f(i) = f(i + 2 \cdot \Delta r) - f(i + \Delta r)$$

This allows us to calculate f as follows:

- If $i \leq r_0$, then this is the aperiodic part of f . We calculated the values $f(1), \dots, f(r_0)$ when we were searching for the first repeat, so just return the respective value.
- If $i > r_0$, then this is the $\lfloor \frac{(i-r_0)}{\Delta r} \rfloor$ -th time the signature has repeated, and the first occurrence of the signature was at $r_0 + ((i - r_0) \bmod \Delta r)$. Then,

$$f(i) = f(r_0 + ((i - r_0) \bmod \Delta r)) + \left\lfloor \frac{(i - r_0)}{\Delta r} \right\rfloor \cdot (f(r_0 + 2 \cdot ((i - r_0) \bmod \Delta r)) - f(r_0 + ((i - r_0) \bmod \Delta r)))$$

This periodicity will allow us to calculate $f(r)$ in $O(c)$ time; we then only have to take into account the pattern q and last row constraint (which both correspond to some element-wise product in the tropical semiring).

So the plan is this: given c and pattern p , calculate the state vector u corresponding to p . (I.e. this vector contains for each two-row-state its cost if the first row matches p , and infinity otherwise.) Then, successively apply M to this vector and keep track of the signatures (and also the corresponding row numbers). Once we hit the same signature twice, we have a cycle / periodicity, and we can compute the rest in $O(c)$ time.

Question is, whether indeed we always have a cycle and whether it is small enough. It turns out that the answer to both is yes, though the best explanation we can give is 'the calculation turns out that way'. There's also the detail regarding handling the degenerate state 111...111; it turns out that since it has a period of 1, and 1 divides Δr , we don't really need to do anything – the formula above

$$f(i) = f(r_0 + ((i - r_0) \bmod \Delta r)) + \left\lfloor \frac{(i - r_0)}{\Delta r} \right\rfloor \cdot (f(r_0 + 2 \cdot ((i - r_0) \bmod \Delta r)) - f(r_0 + ((i - r_0) \bmod \Delta r)))$$

takes care of it automatically.